

LabReSiD26 – Hands-On 12

Davide Ferrara – 518629

1 Esercizio 1: La shell base

L'esercizio richiede di implementare un ciclo *read-fork-exec-wait*: leggere un comando da `stdin`, fare `fork()`, eseguire il programma nel figlio con `execvp()` e aspettarne la terminazione nel padre con `waitpid()`.

Prompt e lettura

Il prompt viene stampato con `printf` seguito da `fflush(stdout)`: `printf` usa un buffer *line-buffered* e, poiché `"minish> "` non termina con `'\n'`, senza `fflush` il testo resterebbe nel buffer e non apparirebbe sullo schermo prima che il programma si blocchi su `fgets()`.

Il built-in `exit` viene gestito **prima** di `fork()`: se venisse controllato nel figlio, sarebbe il figlio a uscire e non la shell.

Figlio: `execvp` e padre: `waitpid`

`execvp(argv[0], argv)` sostituisce l'immagine del processo con il programma richiesto cercandolo nel `PATH`. Se ha successo non ritorna mai: il codice dopo è raggiungibile solo in caso di errore, dove si termina con `exit(127)` (convenzione POSIX per *command not found*). Il padre chiama `waitpid(pid, &status, 0)` e decodifica il risultato con le macro `WIFEXITED/WEXITSTATUS` e `WIFSIGNALED/WTERMSIG`.

```
1 if (pid == 0) {
2     /* execvp non ritorna mai in caso di successo.
3     * exit(127): convenzione POSIX per "command not
4     * found". */
5     if (execvp(argv[0], argv) == -1) {
6         perror(argv[0]);
7         exit(127);
8     }
9 } else {
```

```

9      /* waitpid(pid,...) aspetta il figlio specifico. */
10     int status;
11     waitpid(pid, &status, 0);
12     if (WIFEXITED(status) && WEXITSTATUS(status))
13         printf("[Exit %d]\n", WEXITSTATUS(status));
14     if (WIFSIGNALED(status))
15         printf("[Signal %d]\n", WTERMSIG(status));
16 }

```

Listing 1: Figlio ed padre – parti cruciali

```

1 $ ./minish_es1
2 minish> ls -l
3 total 472
4 -rwxr-xr-x 1 dff dff 15784 May 13 crash
5 -rw-r--r-- 1 dff dff 63 May 13 crash.c
6 -rwxr-xr-x 1 dff dff 16600 May 13 minish_es1
7 minish> echo ciao mondo
8 ciao mondo
9 minish> comando_inesistente
10 comando_inesistente: No such file or directory
11 [Exit 127]
12 minish> exit

```

Listing 2: Test Esercizio 1

2 Esercizio 2: Zombie, segnali e cd

L'esercizio estende la shell con tre funzionalità: esecuzione in background con `&`, raccolta asincrona dei figli tramite `SIGCHLD` e il built-in `cd`.

Comandi in background con `&`

Se l'ultimo token della riga è `&`, si rimuove da `argv`, si stampa il PID del figlio e si torna al loop senza chiamare `waitpid()`.

```

1 /* Il loop e' necessario: i segnali dello stesso tipo
2    non si accodano,
3    * un singolo SIGCHLD puo' rappresentare N terminazioni.
4    */
5 static void sigchld_handler(int sig) {
6     (void)sig;
7     while (waitpid(-1, NULL, WNOHANG) > 0)

```

```

6         ;
7     }
8
9     /* In main, prima del loop: */
10    signal(SIGCHLD, &sigchld_handler);
11
12    /* Rilevazione &: */
13    int bg = argc > 0 && !strcmp(argv[argc - 1], "&");
14    if (bg)
15        argv[--argc] = NULL;
16
17    /* Nel padre: */
18    if (bg) { printf("[1] %d\n", pid); continue; }

```

Listing 3: Rilevazione background e handler SIGCHLD

Raccolta asincrona con SIGCHLD

Quando un figlio in background termina, il kernel manda SIGCHLD al padre. L'handler chiama `waitpid(-1, NULL, WNOHANG)` in un loop per raccogliere tutti i figli terminati, evitando i processi zombie.

Senza handler – zombie accumulati:

```

1 dff    73704    S+    ./minish
2 dff    73731    Z+    [sleep] <defunct>
3 dff    73783    Z+    [sleep] <defunct>
4 ...    (15 zombie)

```

Con handler – nessun zombie:

```

1 dff    72437    S+    ./minish
2 dff    73119    S+    grep -E minish|defunct

```

Built-in cd

`chdir()` modifica la directory corrente del processo che la chiama. Se lo facessimo in un figlio, solo la directory del figlio cambierebbe; la shell rimarrebbe nella directory di partenza. Per questo `cd` viene gestito prima della `fork()`, con `continue` per saltarla. Senza argomenti va in `$HOME`.

```

1 if (!strcmp(argv[0], "cd")) {
2     if (argc == 1) {
3         const char *home = getenv("HOME");

```

```

4         if (!home) { fprintf(stderr, "cd: HOME non
           impostata\n"); continue; }
5         if (chdir(home) < 0) perror("cd");
6     } else {
7         if (chdir(argv[1]) < 0) perror("cd");
8     }
9     continue; /* non fare fork */
10 }

```

Listing 4: Built-in cd

3 Esercizio 3: La pipe

L'esercizio estende la shell per supportare `cmd1 | cmd2`: i due comandi vengono eseguiti in due processi figli collegati da una pipe del kernel.

Rilevazione di `|` e split di `argv[]`

Si scorre `argv[]` finché non si trova `|`: si imposta `argv[i] = NULL` per terminare il vettore sinistro e si salva `argv_right = &argv[i+1]` come puntatore al comando destro.

```

1 char **argv_right = NULL;
2 int i = 0;
3 while (i < argc) {
4     if (!strcmp(argv[i], "|")) {
5         argv[i] = NULL;
6         argv_right = &argv[i + 1];
7         break;
8     }
9     i++;
10 }
11
12 if (argv_right != NULL) {
13     char **argv_left = argv;
14     int fd[2];
15     pipe(fd);
16
17     pid_t child1 = fork();
18     if (child1 == 0) {
19         /* stdout -> pipe: l'output va nel buffer invece
           che a schermo */

```

```

20     dup2(fd[1], STDOUT_FILENO);
21     close(fd[0]); close(fd[1]);
22     execvp(argv_left[0], argv_left);
23     exit(127);
24 }
25
26 pid_t child2 = fork();
27 if (child2 == 0) {
28     /* stdin <- pipe: il comando legge l'output di
29        child1 */
29     dup2(fd[0], STDIN_FILENO);
30     close(fd[1]); close(fd[0]);
31     execvp(argv_right[0], argv_right);
32     exit(127);
33 }
34
35 /* Il padre chiude entrambi i capi: se non chiudesse
36    fd[1],
37    * child2 non riceverebbe mai EOF e rimarrebbe
38    bloccato. */
37 close(fd[0]); close(fd[1]);
38 waitpid(child1, NULL, 0);
39 waitpid(child2, NULL, 0);
40 continue;
41 }

```

Listing 5: Ricerca di — e pipeline

```

1 minish> ls -l | grep .c
2 crash.c
3 minish.c
4 minish_es1.c
5 minish_es2.c
6 minish> cat /etc/passwd | grep root
7 root:x:0:0::/root:/usr/bin/bash
8 minish> echo uno due tre | wc -w
9 3
10 minish> ps aux | grep minish
11 dff  110138  0.0  0.0  2752  1792 pts/3  S+   23:18   0:00
    ./minish
12 dff  111198  0.0  0.0  6676  4212 pts/3  S+   23:24   0:00
    grep minish

```

Listing 6: Test Esercizio 3 – pipe